

AnchorOPT: Towards Optimizing Dynamic Anchors for Adaptive Prompt Learning

Zheng Li¹, Yibing Song, Xin Zhang¹, Lei Luo², Xiang Li^{1*}, Jian Yang^{1*}

¹ Nankai University, ² Nanjing University of Science and Technology

zhengli97@foxmail.com, {xiang.li,implus, csjyang}@nankai.edu.cn

Abstract

Existing prompt learning methods, which are built upon CLIP models, leverage textual tokens as anchors to guide the learnable soft tokens. This guidance improves CLIP generalizations. However, these anchors—static in both value and position—lack cross-task and stage-adaptive flexibility. To address this limitation, we propose AnchorOPT, a dynamic anchor-based prompt learning framework. Specifically, AnchorOPT introduces dynamism in two key dimensions: (i) anchor values eschew handcrafted explicit textual tokens (e.g., “shape”, “color”), instead learning dynamically from task-specific data; and (ii) the positional relationship between anchor and soft tokens is no longer fixed but adaptively optimized via a learnable position matrix conditioned on the training stage and task context. Training occurs in two stages: we first learn the anchor tokens, then freeze and transfer them to the second stage for optimization of soft tokens and the position matrix. Extensive experiments demonstrate that using only a simple learnable anchor and position matrix achieves performance comparable to or exceeding some methods incorporating additional learnable modules or regularization techniques. As a plug-and-play module, AnchorOPT integrates seamlessly into existing frameworks, yielding consistent performance gains across diverse datasets. Code is publicly available at <https://github.com/zhengli97/ATPrompt>.

1. Introduction

Vision-Language Models (VLMs) [15, 32, 37, 38], such as CLIP [32] and ALIGN [15], trained on large-scale web data, have achieved state-of-the-art zero-shot performance on downstream tasks. These models are pre-trained using a contrastive loss to align image and text modalities into a unified embedding space, enabling cross-modal semantic alignment. To further enhance the utilization of pre-trained VLMs, prompt learning [48] has emerged as a crit-

ical methodology for adapting existing models to downstream tasks through the fine-tuning of supplemental soft tokens. CoOp [48] first proposes to replace the fixed handcrafted template (e.g., “A photo of a {Class}.”) with multiple continuous soft tokens as the input of the text encoder, as shown in Fig. 1(b). Subsequent research efforts have attempted to extend textual prompt learning to the vision [1, 30] and multimodal [18, 24] domains, aiming to enhance the coherence of representations across modalities.

Existing mainstream textual-based prompt learning methods [18, 22, 24, 42, 47, 48] primarily adopt an approach that combines multiple learnable soft tokens with fixed class tokens for optimization. However, this paradigm restricts soft tokens to learning only category-specific information derived from training data, thereby limiting their capacity to capture generalizable, category-invariant representations [19, 43, 49]. To address this limitation, recent studies [17, 34, 39] have introduced supplementary information into existing prompts. A common strategy [4, 25, 39, 46] involves inserting key tokens as anchors into input prompts to guide soft tokens toward learning domain-invariant general representations and bridging the gap between base and novel classes [25]. These anchor tokens are typically derived from explicit class attributes [4, 25, 39]. For instance, ATPrompt [25] uses the attributes mined by NAS as anchors to form a novel attribute-based prompt learning template. It can be integrated as a plug-in module in existing methods and has achieved extensive improvements.

However, these textual anchors face two key limitations: manual content curation and positional rigidity, as illustrated in Fig. 1(c). First, anchor tokens are typically manually defined with explicit textual content, embedding human priors that may conflict with domain-specific requirements. Second, while humans dynamically optimize sentence structures to suit contextual demands, existing methods rigidly fix the token positions within prompts, neglecting the need for structural adaptation across tasks.

To address these limitations, we propose AnchorOPT, a dynamic anchor-based prompt learning framework for CLIP. It features two innovations: (1) Dynamic anchor

*Corresponding author.

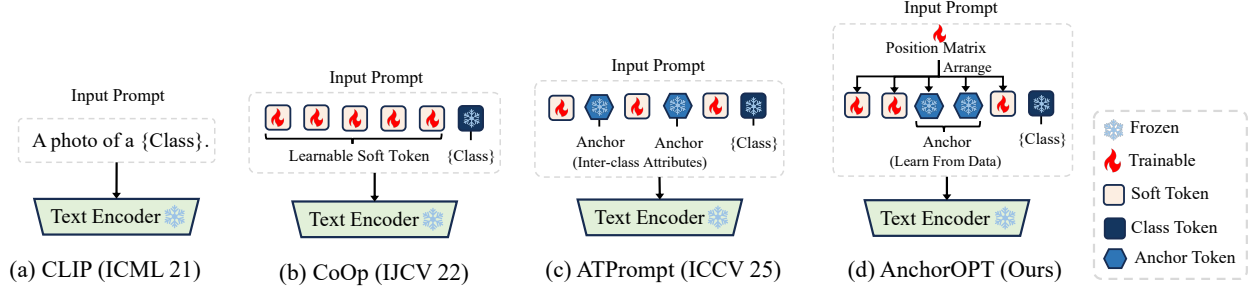


Figure 1. Comparison of fundamental input prompt structures. (a) CLIP[32] employs manually designed text templates. (b) CoOp[48] introduced prompt learning for CLIP adaptation using learnable soft tokens concatenated with fixed category tokens. (c) ATPrompt [25] incorporates explicit, fixed attribute tokens (e.g., “color”, “shape”) to guide soft token learning via an attribute-based template. (d) AnchorOPT utilizes **implicit anchors** learned from data to guide the learning of soft tokens and proposes a learnable position matrix that dynamically adjusts the prompt sequence according to downstream requirements.

values: Explicit static anchors are converted into implicit, learnable tokens that acquire universal cross-category representations directly from data rather than manual definition. (2) Dynamic positional configuration: A learnable position matrix adaptively reconfigures token positions via Gumbel-Softmax [14] optimization, conditioned on task context.

Specifically, our training consists of two distinct stages: the anchor optimization stage and the adaptation stage. In the first stage, we propose a customized anchor learning template: “{*Anc*} of {*Class*}”, where {*Anc*} represents the learnable anchor token. The preposition “of” is used to associate the belonging relationship between the anchor and the class. Through the supervision of the broad category description text, the anchor is encouraged to learn the common internal representation across categories. In the following stage, anchors are frozen and integrated into the normal learnable prompt, then jointly optimized with the position matrix for downstream tasks.

To ensure compatibility with diverse architectures, we propose two depth-adaptive variants (shallow/deep). As a plug-and-play module, AnchorOPT can seamlessly replace existing templates (e.g., CoOp [48], ATPrompt [25]), yielding consistent gains across multiple benchmarks. Notably, AnchorOPT achieves superior baseline performance compared to other conventional templates, reaching performance comparable to or better than many methods [18, 26, 42, 43, 47], which incorporate additional regularization or learnable modules. This demonstrates significant underutilized potential in fundamental prompt structures.

In summary, the contributions are listed as follows:

- We introduce AnchorOPT, a dynamic anchor-based prompt learning framework for CLIP that transforms anchors from explicit static tokens into implicitly optimized representations.
- A learnable position matrix is introduced to enable task-specific prompt restructuring via differentiable token reordering.
- Two depth-compatible variants (shallow/deep) are intro-

duced to facilitate integration with various prompt learning architectures.

- Extensive experiments demonstrate the effectiveness of our AnchorOPT on various datasets.

2. Related Work

Prompt Learning for VLMs. Prompt learning [18, 19, 21, 24, 47, 48] has emerged as a parameter-efficient tool [8, 13] to adapt pre-trained VLMs to downstream tasks. This approach adds a small number of learnable embeddings alongside model input, which are optimized during training while keeping the encoder parameters frozen. After training, models can achieve performance parity with, or even outperform, fully fine-tuned ones [16, 48]. CoOp [48] is the pioneering method for CLIP, which proposed replacing the hand-crafted hard prompt with learnable soft prompts for fine-tuning while freezing the entire pre-trained parameters. Subsequent works have adopted this prompt format and designed a variety of learning methods. CoCoOp [47] aims to enhance the representations of soft tokens by utilizing input images. MaPLe [18] appends the soft tokens to the intermediate hidden representations in both the text and the image encoders. RPO [21] treats the learnable soft tokens of the two modalities as separate classification tokens and averages them to obtain the final prediction result. DePT [45] decouples the representations of base classes and new classes, introducing additional modules to enhance each. PromptKD [24] leverages a large pre-trained teacher model to provide pseudo labels to supervise the CLIP student with learnable prompts on unlabeled domain data.

Prompt Learning with Additional Anchors. Existing prompt learning methods [18, 47, 48] primarily build upon the cascade structure of multiple soft tokens and class tokens proposed by CoOp [48], which serves as input to the encoder during optimization. However, ATPrompt [25] observes that this structure confines soft tokens to learning category-specific representations, thereby disrupting their association with unknown categories. Several studies [25,

[39, 44, 46] have introduced additional tokens as anchors to guide soft tokens toward learning more generalizable representations. These anchor tokens are typically derived from text embeddings. For instance, ArGue [39] builds a pool of intra-class attributes by querying an LLM and appends each attribute to the input prompt for joint optimization, aligning text embeddings with attribute groups. TCP [44] proposes embedding tokens encoded by hand-crafted templates as anchors in the text encoder’s intermediate layers. While previous methods focused on intermediate layers with complex designs, ATPrompt [25] adopts a fundamental approach by proposing attribute-based prompt learning templates that utilize individual attributes as anchor terms, effectively enhancing model generalization. This refocuses research attention on reconsidering the basic forms of prompt learning.

However, these textual anchors exhibit limitations: their content relies on human-defined attributes, and their position remains fixed, overlooking the need for dynamic adaptation of input structure across different tasks or learning stages. To address these issues, we propose AnchorOPT, a dynamic anchor-based prompt learning framework. Specifically, AnchorOPT converts explicit, fixed anchor tokens into implicitly learnable components and introduces a learnable position matrix to dynamically adjust token ordering based on training progress.

3. Method

3.1. Preliminary

Vision-Language Models. Existing vision-language models like CLIP [32] and ALIGN [15] typically have two parallel encoders, one for image and the other for text, which are denoted as $f_{Img}(\cdot)$ and $f_T(\cdot)$, respectively. Let $D = \{(x_i, y_i)\}_{i=1}^I$ be the training dataset, where x_i is the i -th input data sample, y_i is the corresponding ground truth label, I is the size of dataset and $y_i \in \{1, 2, \dots, C\}$ for C -class classification problem. For each input image x , it is first split into P patches, which are then projected to create patch tokens. The image encoder encodes the input patch through the transformer block to produce the visual features $u = f_I(x)/\|f_I(x)\|_2 \in \mathbb{R}^d$, where d represents the feature dimension. On the text side, the text label y and the associated name are formulated with the template, e.g., $t_c = \text{“A photo of a \{Class } c\}”$. The text encoder then encodes the input text through the transformer block, generating the textual feature $v_c = f_T(t_c)/\|f_T(t_c)\|_2 \in \mathbb{R}^d$. Finally, the probability of c -th class for x_i is computed as:

$$q(c|x_i) = \frac{\exp(uv_c^T/\tau)}{\sum_{j=1}^C \exp(uv_j^T/\tau)}. \quad (1)$$

where τ denotes the temperature parameter.

Prompt Learning. Traditional methods often rely on the generic template as a text prompt for calculation. While

this approach works for basic tasks, it lacks specificity for domain- or task-specific applications, failing to yield accurate predictions due to its overgeneralized formulation. Recent works [47, 48] propose to replace fixed text with implicitly learnable soft tokens, which are optimized for downstream tasks while keeping the encoder parameters frozen. Specifically, M soft tokens $[V_1], \dots, [V_M]$ are concatenated with fixed class token $[\text{CLS}]$ and input into the text encoder, as shown in Fig. 1 (a). Its form can be expressed as follows:

$$t_v = [V_1][V_2] \dots [V_M][\text{CLS}]. \quad (2)$$

where M represents the soft prompt length. We omit the prefix and suffix tokens for simplicity.

Subsequently, anchor-based methods [4, 25, 39, 46] were proposed to integrate additional tokens into prompts at fixed positions as anchors, guiding soft tokens to learn general and category-invariant representations. Anchor tokens are typically derived from explicit textual patterns, such as class attributes, which are encoded into the token values. Taking ATPrompt [25] as an example, its prompt structure is formulated as follows:

$$t_{attr} = [V_{a_1}] \dots [V_{a_m}][\text{Anc}^e][V_1] \dots [V_M][\text{CLS}]. \quad (3)$$

where $[\text{Anc}^e]$ represents the explicitly encoded anchor token obtained and $[V]$ represent the learnable soft tokens.

3.2. Prompt Learning with Dynamic Anchor Token

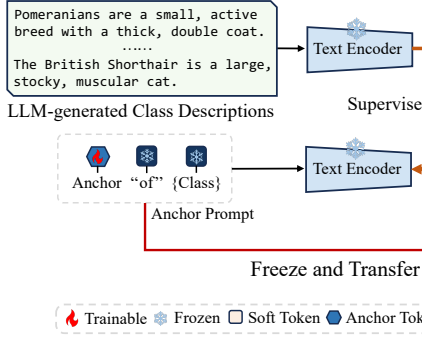
Existing anchor-based prompt learning methods like ATPrompt rely on explicitly encoding text (e.g., “shape”, “color”) as anchor values and fixing their positions within input prompts. This approach imposes human-defined explicit text as token values, which may not best meet the actual needs of downstream tasks. Furthermore, the static nature of these anchors—both in value and position—prevents adaptive refinement during training, limiting their ability to accommodate task-specific needs.

To address these limitations, we propose AnchorOPT, a dynamic anchor-based method for prompt learning, as illustrated in Fig. 1(d). The dynamic nature of anchor tokens manifests in two key aspects: (1) *Value Dynamics*: Anchor token values are learned dynamically from data rather than being manually predefined and kept fixed throughout. (2) *Position Dynamics*: The positions of both soft and anchor tokens are adaptively adjusted through a learnable position matrix, instead of remaining fixed during training. To effectively train and utilize these anchors, we propose a two-stage framework comprising anchor optimization and adaptation. Each stage incorporates the corresponding dynamic aspect. Detailed explanations of both stages follow.

3.2.1. Stage I: Anchor Optimization

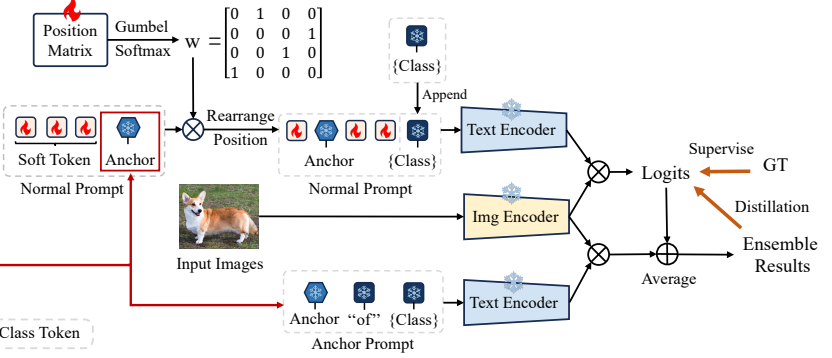
Dynamic Values. Prior work [25] demonstrates that anchors encoding explicit cross-category attributes (e.g.,

Stage I : Train Anchor Token.



(a) Anchor Optimization Stage.

Stage II: Freeze Anchor Token, Train Soft Token & Position Matrix.



(b) Adaptation Stage.

Figure 2. The dynamic anchor token training process comprises two stages: (a) Anchor Optimization: Anchor tokens are initialized as learnable parameters and optimized using LLM-generated category descriptions. The resulting anchors are frozen for the subsequent stage. (b) Adaptation: Soft prompts and the position matrix are jointly optimized for downstream tasks, with knowledge distillation from ensemble results providing auxiliary supervision.

“shape”, “color”) bridge generalization gaps between known and unknown classes. Extending this principle, we propose a specialized anchor prompt template: $t_{anc} = \{ \{Anc\} \text{ of } \{Class\} \}$, where learnable anchor tokens implicitly aggregate cross-category semantics. The preposition “of” models associative relationships between anchors and classes. Specifically, the anchor prompt can be formalized as:

$$t_{anc} = [Anc_1^i] \dots [Anc_N^i] [of] [CLS], \quad (4)$$

where $[Anc^i]$ represents the implicit learnable anchor token, N is the anchor token count, $[of]$ is the fixed preposition embedding, and $[CLS]$ represents the class token. This transforms anchors from explicitly defined static tokens into implicitly optimized representations.

Training. To enable anchor tokens to capture universal cross-category representations, we align the anchor prompt t_{anc} with LLM-generated category descriptions t_d (Fig. 2(a)) using Mean Squared Error (MSE) loss. The objective function can be formulated as:

$$L_{\theta_a} = \text{MSE}(f_T(t_{anc}; \theta_a), f_T(t_d)). \quad (5)$$

where θ_a denotes the anchor token parameters and $f_T(\cdot)$ is the text encoder. Following anchor optimization, the N -token anchor sequence is concatenated with an M -token soft prompt to form the integrated prompt for downstream adaptation.

3.2.2. Stage II: Adaptation

Dynamic Positions. Inspired by how humans adapt sentence structures to meet communicative needs across different contexts, we argue that the fixed structures used in prior work may constrain model adaptability to downstream tasks. To enable adaptive sequence adjustment of the nor-

mal prompt, we introduce a learnable two-dimensional position matrix W of size $(M + N, M + N)$ which dynamically determines the relative positions between soft tokens and anchor tokens, as shown in Fig. 2(b). Here, W_i^j indicates the probability that the j -th token in the original sequence is assigned to the i -th position in the rearranged sequence. However, since the assignment operation is inherently discrete, the matrix W is non-differentiable.

To enable differentiable learning, we adopt the Gumbel-Softmax relaxation [14]. Specifically, for each target position i in the output sequence, we define a probability vector $p^i = [p_1^i, \dots, p_{N+M}^i]$, where p_j^i denotes the probability that token j is selected for position i . During the forward pass, the discrete assignment is obtained by applying the argmax operation with Gumbel noise:

$$\mathbf{w}^i = \text{one_hot}\{\arg \max_j (\log p_j^i + \epsilon_j)\}, \quad (6)$$

where $\epsilon_j \sim \text{Gumbel}(0, 1)$ are independent samples from the standard Gumbel distribution, and $\text{one_hot}(\cdot)$ converts the index into a one-hot vector.

During training, to allow gradient backpropagation, we approximate the discrete assignment using a continuous softmax:

$$\mathbf{w}_j^i = \frac{\exp((\log p_j^i + \epsilon_j)/\tau)}{\sum_{k=1}^{N+M} \exp((\log p_k^i + \epsilon_k)/\tau)}. \quad (7)$$

where the temperature parameter τ controls the softmax sharpness. The resulting softened weight matrix $\hat{W}$ is used for gradient computation, while the hard assignment version is applied during inference.

Subsequently, the frozen anchor tokens are concatenated with soft tokens to form the composite prompt t_v , which is

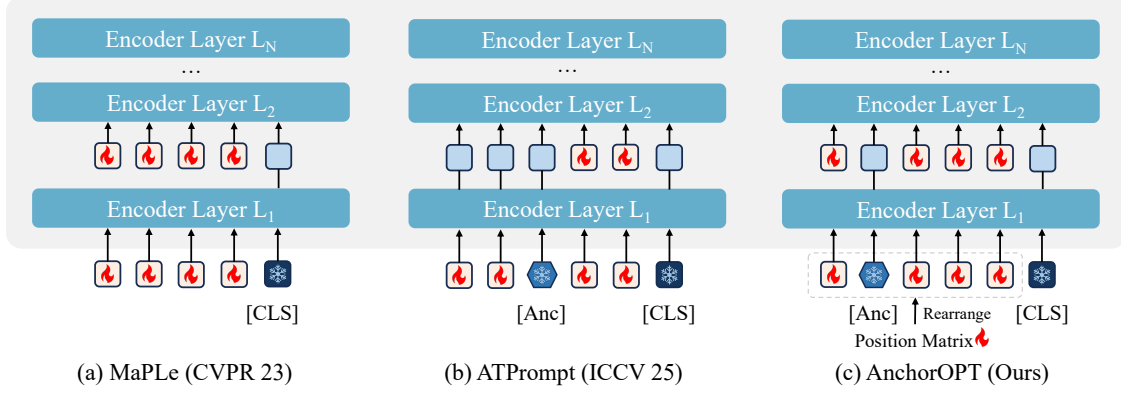


Figure 3. Computational process in deep prompt learning variants: (a) MaPLe drops and reintroduces all soft tokens after each Transformer block. (b) ATPrompt retains all attribute-related hard/soft tokens while discarding class-related soft tokens. (c) AnchorOPT dynamically reorders tokens via the position matrix, retaining only anchor tokens and discarding all soft tokens during processing.

then restructured via the position matrix before appending the class token [CLS] for encoder input. This process can be formally expressed as:

$$t_v = [V_1] \dots [V_M] [\text{Anc}_1^i] \dots [\text{Anc}_N^i], \quad (8)$$

$$t_{norm} = \text{concat}(\hat{W} \odot t_v, [\text{CLS}]). \quad (9)$$

The resulting t_{norm} is then passed to the text encoder as a structured prompt for downstream processing.

Training. In the second stage, two prompts are used: the trainable normal prompt t_{norm} and the pre-trained anchor prompt t_{anc} from the first stage. For the normal prompt, we optimize the learnable position matrix and soft tokens using the conventional cross-entropy loss. Let θ_s and θ_{pos} denote the parameters of the soft tokens and position matrix, respectively. The loss function $L_{\theta_s, \theta_{pos}}$ is formulated as:

$$L_{\theta_s, \theta_{pos}}^1 = \text{CE}(q_{norm}, y), \quad (10)$$

where $q_{norm} = f(x, t_{norm}; \theta_s, \theta_p)$ denotes the output of the CLIP model given image x and prompt t_{norm} .

For the anchor prompt, we first compute its prediction q_{anc} for the input image. This prediction is then combined with the normal prompt's output q_{norm} through averaging to form an ensemble prediction q_{ens} . We then distill the knowledge [12, 23, 50] from this ensemble output to the learnable normal prompt using the KL divergence loss:

$$L_{\theta_s, \theta_{pos}}^2 = \text{KL}(q_{norm}, q_{ens}), \quad (11)$$

where $\text{KL}(\cdot, \cdot)$ denotes the Kullback–Leibler divergence.

The overall training objective for the second stage is:

$$L_{total} = \lambda_1 \cdot L_{\theta_s, \theta_{pos}}^1 + \lambda_2 \cdot L_{\theta_s, \theta_{pos}}^2. \quad (12)$$

where λ_1 and λ_2 are hyperparameters that control the loss balance, typically set to a 1:10 ratio.

Inference. During inference, we use predictions from well-trained normal prompts for known base classes, while employing ensemble results for unknown novel classes.

3.2.3. Deep Version of AnchorOPT

Beyond incorporating soft tokens solely in the input layer (shallow variant), we extend this approach by introducing learnable soft tokens in deeper transformer layers, as depicted in Fig. 3(c). Initially, a position matrix dynamically reorders soft and anchor tokens, which are then concatenated with the class token for network input. Unlike existing methods [18, 19, 24] that discard all soft tokens after each transformer block's forward pass and reintroduce them before subsequent blocks, **our method preserves anchor tokens throughout deep-layer processing to maintain critical semantic guidance.** This retention enables anchor tokens to influence deeper representations while soft tokens are selectively discarded and reintroduced between layers.

3.2.4. One-Stage Training Extension

While our primary framework employs a two-stage training paradigm that decouples anchor tokens from task-specific soft tokens, we propose a computationally efficient one-stage co-training paradigm. This unified approach integrates both training phases into alternating optimization steps within a single framework: (1) updating anchor tokens, and (2) freezing anchor parameters while optimizing soft tokens and the position matrix. As unstable intermediate ensemble results cannot yield reliable supervision signals, we eliminate the distillation objective during one-stage training. More training details are presented in the Appendix.

4. Experiments

4.1. Settings

Baseline Methods. We select four influential prompt learning methods as our baseline and backbone models for our plug-and-play technique, including CoOp [48], Co-CoOp [47], MaPLe [18], and DePT [45]. In addition, we also list other methods for comparison, including Prompt-

Method	Average			ImageNet			Caltech101			OxfordPets		
	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM
PromptSRC (ICCV 23)	84.26	76.10	79.97	77.60	70.73	74.01	98.10	94.03	96.02	95.33	97.30	96.30
CoPrompt (ICLR 24)	84.00	77.23	80.48	77.67	71.27	74.33	98.27	94.90	96.55	95.67	98.10	96.87
TCP (CVPR 24)	84.13	75.36	79.51	77.27	69.87	73.38	98.23	94.67	96.42	94.67	97.20	95.92
TextRefiner (AAAI 25)	79.74	74.32	76.94	76.84	70.54	73.56	98.13	94.43	96.24	95.27	97.65	96.45
DPC (CVPR 25)	85.15	68.84	76.13	77.72	68.85	73.02	98.58	94.65	96.58	95.80	97.60	96.69
CoOp (IJCV 22)	82.69	63.22	71.66	76.47	67.88	71.92	98.00	89.81	93.73	93.67	95.29	94.47
+ ATPrompt (ICCV 25)	82.68	68.04	74.65 (+2.99)	76.27	70.60	73.33	97.95	93.63	95.74	94.77	96.59	95.67
+ AnchorOPT (Ours)	81.24	76.27	78.68 (+7.02)	76.30	71.56	73.85	98.26	95.34	96.78	95.32	98.04	96.66
CoCoOp (CVPR 22)	80.47	71.69	75.83	75.98	70.43	73.10	97.96	93.81	95.84	95.20	97.69	96.43
+ ATPrompt (ICCV 25)	81.69	74.54	77.95 (+2.21)	76.43	70.50	73.35	97.96	95.27	96.60	95.46	97.89	96.66
+ AnchorOPT (Ours)	81.87	77.06	79.39 (+3.56)	76.32	71.78	73.98	98.17	95.56	96.85	95.87	98.06	96.95
MaPLe (CVPR 23)	82.28	75.14	78.55	76.66	70.54	73.47	97.74	94.36	96.02	95.43	97.76	96.58
+ ATPrompt (ICCV 25)	82.98	75.76	79.21 (+0.66)	76.94	70.72	73.70	98.32	95.09	96.68	95.62	97.63	96.61
+ AnchorOPT (Ours)	83.62	77.36	80.37 (+1.82)	76.98	71.88	74.34	97.87	96.03	96.94	96.38	98.23	97.30
DePT (CVPR 24)	83.66	71.82	77.29	77.13	70.10	73.45	98.33	94.33	96.29	94.70	97.63	96.14
+ ATPrompt (ICCV 25)	83.80	73.75	78.45 (+1.16)	77.32	70.65	73.83	98.48	94.60	96.50	94.65	97.99	96.29
+ AnchorOPT (Ours)	84.27	76.90	80.42 (+3.13)	77.56	71.67	74.50	98.20	95.78	96.97	95.39	98.23	96.79

Method	StanfordCars			Flowers102			Food101			FGVCAircraft		
	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM
PromptSRC (ICCV 23)	78.27	74.97	76.58	98.07	76.50	85.95	90.67	91.53	91.10	42.73	37.87	40.15
CoPrompt (ICLR 24)	76.97	74.40	75.66	97.27	76.60	85.71	90.73	92.07	91.40	40.20	39.33	39.76
TCP (CVPR 24)	80.80	74.13	77.32	97.73	75.57	85.23	90.57	91.37	90.97	41.97	34.43	37.83
TextRefiner (AAAI 25)	71.40	70.90	71.15	95.92	74.33	83.76	90.88	91.43	91.15	35.35	35.87	35.61
DPC (CVPR 25)	81.13	70.14	75.24	98.86	68.37	80.84	91.15	91.47	91.31	45.56	24.24	31.64
CoOp (IJCV 22)	78.12	60.40	68.13	97.60	59.67	74.06	88.33	82.26	85.19	40.44	22.30	28.75
+ ATPrompt (ICCV 25)	77.43	66.55	71.58	97.44	67.52	79.77	88.74	87.44	88.09	40.38	27.22	32.52
+ AnchorOPT (Ours)	76.73	74.43	75.67	97.22	75.41	84.94	90.40	91.95	91.17	38.48	35.47	36.91
CoCoOp (CVPR 22)	70.49	73.59	72.01	94.87	71.75	81.71	90.70	91.29	90.99	33.41	23.71	27.74
+ ATPrompt (ICCV 25)	74.50	73.47	73.98	96.52	73.59	83.51	90.59	91.74	91.16	37.30	33.15	35.10
+ AnchorOPT (Ours)	75.59	75.23	75.41	96.17	77.35	85.74	90.58	91.98	91.27	36.90	36.35	36.62
MaPLe (CVPR 23)	72.94	74.00	73.47	95.92	72.46	82.56	90.71	92.05	91.38	37.44	35.61	36.50
+ ATPrompt (ICCV 25)	75.39	73.84	74.61	97.82	75.07	84.95	90.65	92.00	91.32	37.61	36.15	36.87
+ AnchorOPT (Ours)	77.30	74.49	75.87	97.79	77.35	86.38	90.58	92.16	91.36	38.72	38.19	38.45
DePT (CVPR 24)	79.67	72.40	75.86	98.20	72.00	83.08	90.43	91.33	90.88	42.53	22.53	29.46
+ ATPrompt (ICCV 25)	79.29	73.47	76.27	98.20	73.69	84.20	90.42	91.69	91.05	43.19	33.23	37.56
+ AnchorOPT (Ours)	80.98	75.63	78.21	98.10	78.30	87.09	90.53	91.93	91.22	44.34	36.09	39.79

Method	SUN397			DTD			EuroSAT			UCF101		
	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM
PromptSRC (ICCV 23)	82.67	78.47	80.52	83.37	62.97	71.75	92.90	73.90	82.32	87.10	78.80	82.74
CoPrompt (ICLR 24)	82.63	80.03	81.30	83.13	64.73	72.79	94.60	78.57	85.84	86.90	79.57	83.07
TCP (CVPR 24)	82.63	78.20	80.35	82.77	58.07	68.25	91.63	74.73	82.32	87.13	80.77	83.83
TextRefiner (AAAI 25)	80.96	76.49	78.66	75.35	58.09	65.60	74.57	72.82	73.68	82.52	75.01	78.59
DPC (CVPR 25)	82.81	74.10	78.21	84.61	49.88	62.76	93.40	51.62	66.49	87.02	66.31	75.27
CoOp (IJCV 22)	80.60	65.89	72.51	79.44	41.18	54.24	92.19	54.74	68.69	84.69	56.05	67.46
+ ATPrompt (ICCV 25)	80.84	68.64	74.24	80.83	45.49	58.22	90.34	59.79	71.96	84.49	64.96	73.45
+ AnchorOPT (Ours)	79.67	78.73	79.20	74.04	61.72	67.32	83.75	77.14	80.31	83.45	79.18	81.26
+ CoCoOp (CVPR 22)	79.74	76.86	78.27	77.01	56.00	64.85	87.49	60.04	71.21	82.33	73.45	77.64
+ ATPrompt (ICCV 25)	80.50	76.86	78.64	78.63	56.89	66.02	87.95	74.15	80.46	82.74	76.40	79.44
+ AnchorOPT (Ours)	79.74	79.25	79.49	77.43	63.29	69.65	89.54	79.51	84.23	83.27	79.28	81.23
MaPLe (CVPR 23)	80.82	78.70	79.75	80.36	59.18	68.16	94.07	73.23	82.35	83.00	78.66	80.77
+ ATPrompt (ICCV 25)	80.98	78.15	79.54	80.50	58.28	67.61	94.84	77.59	85.35	84.08	78.88	81.40
+ AnchorOPT (Ours)	81.99	79.57	80.76	81.94	60.94	69.90	95.92	81.63	88.20	84.35	80.53	82.40
DePT (CVPR 24)	82.37	75.07	78.55	83.20	56.13	67.04	88.27	66.27	75.70	85.43	72.17	78.24
+ ATPrompt (ICCV 25)	82.42	76.48	79.34	82.64	56.77	67.30	89.60	69.50	78.28	85.60	73.15	78.89
+ AnchorOPT (Ours)	82.41	79.08	80.71	81.87	63.16	71.31	91.12	77.80	83.93	86.42	78.27	82.14

Table 1. Base-to-novel generalization experiments of various baselines with and without AnchorOPT on 11 datasets. As a foundational template-based method, AnchorOPT demonstrates superior performance compared to ATPrompt. Notably, equipped with our AnchorOPT, the current simple method exhibits robust capabilities, even surpassing some complex, advanced methods that introduce additional regularization techniques or learning modules.

Method	Source	Target Dataset										
	Image Net	Caltech 101	Oxford Pets	Stanford Cars	Flowers 102	Food 101	FGVC Aircraft	SUN 397	DTD	Euro SAT	UCF 101	Average
CoCoOp	71.02	94.43	90.14	65.32	71.88	86.06	22.94	67.36	45.73	45.37	68.21	65.74
MaPLe	70.72	93.53	90.49	65.57	72.23	86.20	24.74	67.01	46.49	48.06	68.69	66.30
TCP	71.40	93.97	91.25	64.69	71.21	86.69	23.45	67.15	44.35	51.45	68.73	66.29
CoPrompt	70.80	94.50	90.73	65.67	72.30	86.43	24.00	67.57	47.07	51.90	69.73	67.00
CoOp	71.51	93.70	89.14	64.51	68.71	85.30	18.47	64.15	41.92	46.39	66.55	63.88
+ ATPrompt	71.67	93.96	90.65	65.01	70.40	85.86	20.97	65.77	43.44	46.59	69.92	65.26 (+1.38)
+ AnchorOPT	71.33	94.65	90.08	65.90	72.39	86.51	23.13	67.99	46.93	52.10	69.60	66.93 (+3.05)

Table 2. Cross-dataset generalization experiments on ten datasets. AnchorOPT achieves average performance improvement on target datasets. It’s worth noting that simply combining AnchorOPT with CoOp achieves performance comparable to more complex methods like CoPrompt, indicating significant untapped potential in designing basic prompting learning templates.

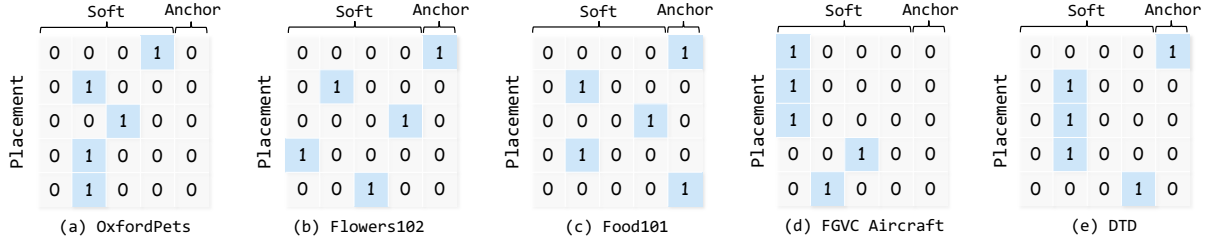


Figure 4. Position matrix visualizations across five datasets. For the Oxford Pets dataset, a value of 1 at row 1, column 4 indicates that token 4 in the original sequence is mapped to position 1 in the transformed sequence. Each dataset exhibits distinct convergence patterns. Note that all-zero values in the final column of the position matrix do not imply anchor tokens are redundant; though omitted from the visualization, these tokens contribute to intermediate computation stages during training.

SRC [19], CoPrompt [35], TCP [44], TextRefiner [42], and DPC [22].

Implementation Details. All experiments were conducted on a single NVIDIA H20 GPU. We use the ViT-B/16 CLIP as our default model. The length of soft tokens is usually set to 4 or 6. The length of the anchor token is set to 1. We report base and novel class accuracy and harmonic mean averaged over 3 runs. Complete details are attached in the Appendix.

4.2. Base-to-Novel Generalization

Results. In Table 1, we report the quantitative results for base-to-novel tasks on 11 diverse datasets. It shows that, combined with our AnchorOPT method, multiple baseline methods have achieved consistent improvements, ranging from 1.82% to 7.02%. As a fundamental template-based method, AnchorOPT exhibits superior performance compared to ATPrompt.

Position Matrix Visualization. The position matrix learned by our method is visualized in Fig. 4. It reveals two interesting observations: (1) the matrix sometimes repeatedly selects identical tokens during sequence construction, and (2) tokens exhibit varying importance, with certain tokens discarded in the final stage.

Extension to One-stage Training. Table 3 compares the average performance of one-stage and two-stage training paradigms using the CoOp method across 11 datasets. The results indicate that while the one-stage approach entails a

less complex computational process, it yields lower performance than the two-stage paradigm.

Paradigm	Base	Novel	HM
One-stage	80.36	75.89	78.06
Two-stage	81.24	76.27	78.68

Table 3. Comparison of training paradigm on 11 datasets. Two-stage approaches yield better results.

4.3. Cross-dataset Experiment

Table 2 shows the cross-dataset experimental results on various datasets. Compared to ATPrompt, our method demonstrates stronger cross-dataset generalization ability.

4.4. Domain Generalization

Table 4 shows the domain generalization results on four datasets. CoOp+AnchorOPT achieves stronger domain generalization performance than CoPrompt.

4.5. Ablation Study

To minimize the influence of other components in the method, we choose CoOp+AnchorOPT as the baseline. For experiments on ImageNet, we set the anchor length to 1 and the soft token length to 6 by default. Complete experimental results are attached in the Appendix.

Ensemble Operation. In this work, we average predictions from two prompts as the final output for novel classes.

Method	Source	Target Dataset				Average
	IN	-V2	-S	-A	-R	
CoCoOp	71.02	64.07	48.75	50.63	76.18	59.91
MaPLe	70.72	64.07	49.15	50.90	76.98	60.27
TCP	71.20	64.6	49.50	51.2	76.73	60.51
CoPrompt	70.80	64.25	49.43	50.50	77.51	60.42
TextRefiner	72.06	65.02	48.58	49.77	76.30	59.92
CoOp	71.51	64.20	47.99	49.71	75.21	59.28
+ ATPrompt	71.67	64.43	49.13	50.91	76.24	60.18 (+0.90)
+ AnchorOPT	71.33	64.37	49.81	51.64	77.32	60.79 (+1.51)

Table 4. Domain generalization experiments on four datasets. The integration of AnchorOPT resulted in better performance.

Table 5 compares the generalization performance of AnchorOPT’s learnable prompts against ATPrompt excluding anchor prompts. Our normal prompts outperform ATPrompt even without anchor integration, demonstrating superior adaptability.

Method	Base	Novel	HM
ATPrompt	82.68	68.04	74.65
AnchorOPT w/o Ensemble	81.24	72.46	76.60

Table 5. Comparison of learnable normal prompt performance on 11 datasets. Even without the integration of anchor prompts, our method still outperforms previous work.

Preposition “of”. We employ the preposition “of” to construct anchor prompts because it can establish a possessive relationship between anchor and class tokens, aligning with our training objectives for anchor tokens. Table 6 evaluates preposition impacts on ImageNet, showing that irrelevant prepositions significantly impair novel-class generalization by failing to guide anchor token training effectively.

Preposition Type	Value	Base	Novel	HM
Similar	“with”	75.96	71.49	73.66
	“at”	76.07	71.54	73.74
Irrelevant	“sun”	75.96	71.05	73.42
	“sea”	75.69	70.89	73.21
Ours	“of”	76.30	71.56	73.85

Table 6. Comparison of different preposition words on ImageNet. The preposition “of” works best.

Anchor Length. With fixed soft token length (6), Table 7 shows that anchor length critically influences performance. An anchor length of 1 yields optimal results, while longer anchors progressively degrade performance due to excessive constraints on soft token adaptability. Notably, while ATPrompt identifies 2–3 explicit attribute anchors as optimal, our method achieves peak performance with a single implicit anchor, indicating greater representational efficiency and robustness in learned anchors.

Anchor Arrangement. Table 8 evaluates token arrangement strategies—adaptive versus fixed positioning—with

Type	Anchor Prompt			Normal Prompt		
Length	Base	Novel	HM	Base	Novel	HM
1	74.63	71.03	72.79	76.30	69.51	72.75
2	74.58	70.82	72.65	72.14	63.32	67.44
3	74.54	71.17	72.82	34.23	11.17	16.84
4	74.54	71.12	72.79	13.41	1.52	2.73

Table 7. Comparison of different anchor lengths on ImageNet. Optimal performance is achieved when the anchor length is 1.

anchor token length fixed at 1. The adaptive arrangement, dynamically adjusted via the position matrix, consistently outperforms fixed-position variants on ImageNet.

Type	Position	Base	Novel	HM
Adaptive	Position Matrix	76.30	71.56	73.85
Fixed	Before Soft Token	76.03	71.27	73.57
	Middle	75.99	71.17	73.50
	After Class Token	76.23	71.31	73.69

Table 8. Comparison of different anchor token arrangements on ImageNet. Using a position matrix yields the best results.

Distillation. Table 9 quantifies the impact of distillation on generalization performance. Incorporating distillation from ensemble predictions enables soft tokens to acquire more generalizable representations for novel classes.

KD	Base	Novel	HM
	75.86	71.36	73.54
✓	76.30	71.56	73.85

Table 9. Ablation of the knowledge distillation strategy on ImageNet. Removing the distillation term weakens overall generalization performance.

5. Conclusion

We propose AnchorOPT, a dynamic anchor-based prompt learning framework that optimizes explicit fixed anchors into implicit learnable tokens and introduces a position matrix for adaptive prompt sequencing. Our method employs a two-stage training paradigm to sequentially optimize anchor tokens and downstream adaptation of soft prompts and position matrix. To ensure compatibility across architectures, we develop shallow and deep variants. Extensive experiments across 11 datasets demonstrate that AnchorOPT, combined with CoOp, achieves competitive performance against more complex methods, revealing significant untapped potential in foundational prompt learning templates. These results indicate that basic prompt structures, since CoOp’s introduction, remain suboptimal and warrant further exploration.

References

- [1] Hyojin Bahng, Ali Jahanian, Swami Sankaranarayanan, and Phillip Isola. Exploring visual prompts for adapting large-scale models. *arXiv preprint arXiv:2203.17274*, 2022. [1](#)
- [2] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101—mining discriminative components with random forests. In *ECCV*, pages 446–461. Springer, 2014. [11](#)
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *NeurIPS*, 33:1877–1901, 2020. [11](#)
- [4] Keyan Chen, Xiaolong Jiang, Yao Hu, Xu Tang, Yan Gao, Jianqi Chen, and Weidi Xie. Ovaret: Towards open-vocabulary object attribute recognition. In *CVPR*, pages 23518–23527, 2023. [1, 3](#)
- [5] Mircea Cimpoi, Subhransu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *CVPR*, pages 3606–3613, 2014. [11](#)
- [6] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, pages 248–255, 2009. [11](#)
- [7] Li Fei-Fei, Rob Fergus, and Pietro Perona. Learning generative visual models from few training examples: An incremental bayesian approach tested on 101 object categories. In *CVPR workshop*, pages 178–178. IEEE, 2004. [11](#)
- [8] Peng Gao, Shijie Geng, Renrui Zhang, Teli Ma, Rongyao Fang, Yongfeng Zhang, Hongsheng Li, and Yu Qiao. Clip-adapter: Better vision-language models with feature adapters. *IJCV*, 132(2):581–595, 2024. [2](#)
- [9] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7):2217–2226, 2019. [11](#)
- [10] Dan Hendrycks, Steven Basart, Norman Mu, Saurav Kadavath, Frank Wang, Evan Dorundo, Rahul Desai, Tyler Zhu, Samyak Parajuli, Mike Guo, et al. The many faces of robustness: A critical analysis of out-of-distribution generalization. In *ICCV*, pages 8340–8349, 2021. [11](#)
- [11] Dan Hendrycks, Kevin Zhao, Steven Basart, Jacob Steinhardt, and Dawn Song. Natural adversarial examples. In *CVPR*, pages 15262–15271, 2021. [11](#)
- [12] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015. [5](#)
- [13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022. [2](#)
- [14] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144*, 2016. [2, 4](#)
- [15] Chao Jia, Yinfei Yang, Ye Xia, Yi-Ting Chen, Zarana Parekh, Hieu Pham, Quoc Le, Yun-Hsuan Sung, Zhen Li, and Tom Duerig. Scaling up visual and vision-language representation learning with noisy text supervision. In *ICML*, pages 4904–4916. PMLR, 2021. [1, 3](#)
- [16] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual prompt tuning. In *ECCV*, pages 709–727. Springer, 2022. [2](#)
- [17] Baoshuo Kan, Teng Wang, Wenpeng Lu, Xiantong Zhen, Weili Guan, and Feng Zheng. Knowledge-aware prompt tuning for generalizable vision-language models. In *ICCV*, pages 15670–15680, 2023. [1](#)
- [18] Muhammad Uzair Khattak, Hanoona Rasheed, Muhammad Maaz, Salman Khan, and Fahad Shahbaz Khan. Maple: Multi-modal prompt learning. In *CVPR*, pages 19113–19122, 2023. [1, 2, 5](#)
- [19] Muhammad Uzair Khattak, Syed Talal Wasim, Muzammal Naseer, Salman Khan, Ming-Hsuan Yang, and Fahad Shahbaz Khan. Self-regulating prompts: Foundational model adaptation without forgetting. In *ICCV*, pages 15190–15200, 2023. [1, 2, 5, 7](#)
- [20] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3d object representations for fine-grained categorization. In *ICCV workshop*, pages 554–561, 2013. [11](#)
- [21] Dongjun Lee, Seokwon Song, Jihee Suh, Joonmyeong Choi, Sanghyeok Lee, and Hyunwoo J Kim. Read-only prompt optimization for vision-language few-shot learning. In *ICCV*, pages 1401–1411, 2023. [2](#)
- [22] Haoyang Li, Liang Wang, Chao Wang, Jing Jiang, Yan Peng, and Guodong Long. Dpc: Dual-prompt collaboration for tuning vision-language models. *arXiv preprint arXiv:2503.13443*, 2025. [1, 7](#)
- [23] Zheng Li, Xiang Li, Lingfeng Yang, Borui Zhao, Renjie Song, Lei Luo, Jun Li, and Jian Yang. Curriculum temperature for knowledge distillation. In *AAAI*, pages 1504–1512, 2023. [5](#)
- [24] Zheng Li, Xiang Li, Xinyi Fu, Xin Zhang, Weiqiang Wang, Shuo Chen, and Jian Yang. Promptkd: Unsupervised prompt distillation for vision-language models. In *CVPR*, pages 26617–26626, 2024. [1, 2, 5](#)
- [25] Zheng Li, Yibing Song, Ming-Ming Cheng, Xiang Li, and Jian Yang. Advancing textual prompt learning with anchored attributes. In *ICCV*, pages 3618–3627, 2025. [1, 2, 3](#)
- [26] Yuning Lu, Jianzhuang Liu, Yonggang Zhang, Yajing Liu, and Xinmei Tian. Prompt distribution learning. In *CVPR*, pages 5206–5215, 2022. [2](#)
- [27] Subhransu Maji, Esa Rahtu, Juho Kannala, Matthew Blaschko, and Andrea Vedaldi. Fine-grained visual classification of aircraft. *arXiv preprint arXiv:1306.5151*, 2013. [11](#)
- [28] Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *2008 Sixth Indian conference on computer vision, graphics & image processing*, pages 722–729. IEEE, 2008. [11](#)
- [29] Omkar M Parkhi, Andrea Vedaldi, Andrew Zisserman, and CV Jawahar. Cats and dogs. In *CVPR*, pages 3498–3505. IEEE, 2012. [11](#)
- [30] Wenjie Pei, Tongqi Xia, Fanglin Chen, Jinsong Li, Jiandong Tian, and Guangming Lu. Sa²vp: Spatially aligned-and-

- adapted visual prompt. In *AAAI*, pages 4450–4458, 2024. [1](#)
- [31] Sarah Pratt, Ian Covert, Rosanne Liu, and Ali Farhadi. What does a platypus look like? generating customized prompts for zero-shot image classification. In *ICCV*, pages 15691–15701, 2023. [11](#)
- [32] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *ICML*, pages 8748–8763. PMLR, 2021. [1](#), [2](#), [3](#)
- [33] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do imagenet classifiers generalize to imagenet? In *ICML*, pages 5389–5400. PMLR, 2019. [11](#)
- [34] Shuhuai Ren, Aston Zhang, Yi Zhu, Shuai Zhang, Shuai Zheng, Mu Li, Alexander J Smola, and Xu Sun. Prompt pre-training with twenty-thousand classes for open-vocabulary visual recognition. *NeurIPS*, 36:12569–12588, 2023. [1](#)
- [35] Shuvendu Roy and Ali Etemad. Consistency-guided prompt learning for vision-language models. *arXiv preprint arXiv:2306.01195*, 2023. [7](#)
- [36] Khurram Soomro, Amir Roshan Zamir, and Mubarak Shah. Ucf101: A dataset of 101 human actions classes from videos in the wild. *arXiv preprint arXiv:1212.0402*, 2012. [11](#)
- [37] Zeyi Sun, Ye Fang, Tong Wu, Pan Zhang, Yuhang Zang, Shu Kong, Yuanjun Xiong, Dahua Lin, and Jiaqi Wang. Alpha-clip: A clip model focusing on wherever you want. In *CVPR*, pages 13019–13029, 2024. [1](#)
- [38] Hao Tan and Mohit Bansal. Lxmert: Learning cross-modality encoder representations from transformers. *arXiv preprint arXiv:1908.07490*, 2019. [1](#)
- [39] Xinyu Tian, Shu Zou, Zhaoyuan Yang, and Jing Zhang. Argue: Attribute-guided prompt tuning for vision-language models. In *CVPR*, pages 28578–28587, 2024. [1](#), [3](#)
- [40] Haohan Wang, Songwei Ge, Zachary Lipton, and Eric P Xing. Learning robust global representations by penalizing local predictive power. *NeurIPS*, 32, 2019. [11](#)
- [41] Jianxiong Xiao, James Hays, Krista A Ehinger, Aude Oliva, and Antonio Torralba. Sun database: Large-scale scene recognition from abbey to zoo. In *CVPR*, pages 3485–3492. IEEE, 2010. [11](#)
- [42] Jingjing Xie, Yuxin Zhang, Jun Peng, Zhaohong Huang, and Liujuan Cao. Textrefiner: Internal visual feature as efficient refiner for vision-language models prompt tuning. In *AAAI*, pages 8718–8726, 2025. [1](#), [2](#), [7](#)
- [43] Hantao Yao, Rui Zhang, and Changsheng Xu. Visual-language prompt tuning with knowledge-guided context optimization. In *CVPR*, pages 6757–6767, 2023. [1](#), [2](#)
- [44] Hantao Yao, Rui Zhang, and Changsheng Xu. Tcpr: Textual-based class-aware prompt tuning for visual-language model. In *CVPR*, pages 23438–23448, 2024. [3](#), [7](#)
- [45] Ji Zhang, Shihan Wu, Lianli Gao, Heng Tao Shen, and Jingkuan Song. Dept: Decoupled prompt tuning. In *CVPR*, pages 12924–12933, 2024. [2](#), [5](#)
- [46] Zhaoheng Zheng, Jingmin Wei, Xuefeng Hu, Haidong Zhu, and Ram Nevatia. Large language models are good prompt learners for low-shot image classification. In *CVPR*, pages 28453–28462, 2024. [1](#), [3](#)
- [47] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Conditional prompt learning for vision-language models. In *CVPR*, pages 16816–16825, 2022. [1](#), [2](#), [3](#), [5](#)
- [48] Kaiyang Zhou, Jingkang Yang, Chen Change Loy, and Ziwei Liu. Learning to prompt for vision-language models. *IJCV*, 130(9):2337–2348, 2022. [1](#), [2](#), [3](#), [5](#)
- [49] Beier Zhu, Yulei Niu, Yucheng Han, Yue Wu, and Hanwang Zhang. Prompt-aligned gradient for prompt tuning. In *ICCV*, pages 15659–15669, 2023. [1](#)
- [50] Xiatian Zhu, Shaogang Gong, et al. Knowledge distillation by on-the-fly native ensemble. In *NeurIPS*, pages 7517–7527, 2018. [5](#)

AnchorOPT: Towards Optimizing Dynamic Anchors for Adaptive Prompt Learning

Supplementary Material

S1. One-stage Training Paradigm Extension

Fig. S1 illustrates the one-stage training paradigm, which unifies the previous two-stage process into sequential optimization steps within a single framework. This approach alternates between optimizing target parameters while freezing non-target parameters, yielding stable convergence. By eliminating stage separation, the paradigm reduces training complexity while maintaining performance integrity. Table S1 shows the results of the one-stage training method on 11 datasets. Our method adopts the two-stage paradigm, which demonstrates two key advantages over one-stage alternatives: (1) enhanced classification accuracy, and (2) streamlined hyperparameter optimization through elimination of redundant anchor token retraining during each training session.

S2. Implementation Details

S2.1. Dataset

We evaluate the performance of our method on 15 recognition datasets. For generalization from base-to-novel classes and cross-dataset evaluation, we evaluate the performance of our method on 11 diverse recognition datasets. Specifically, these datasets include ImageNet-1K [6] and Caltech-101 [7] for generic object classification; OxfordPets [29], Stanford Cars [20], Flowers102 [28], Food101 [2], and FGVC Aircraft [27] for fine-grained classification, SUN-397 [41] for scene recognition, UCF-101 [36] for action recognition, DTD [5] for texture classification, and EuroSAT [9] for satellite imagery recognition. For domain generalization experiments, we use ImageNet-1K [6] as the source dataset and its four variants as target datasets, including ImageNet-V2 [33], ImageNet-Sketch [40], ImageNet-A [11], and ImageNet-R [10].

S2.2. Training Details

All experiments were conducted on a single NVIDIA H20 GPU. We use the ViT-B/16 CLIP as our default model. We adopted the standard data augmentation scheme as the baseline method, including random resized cropping and flipping. We employed Stochastic Gradient Descent (SGD) as the optimizer for both soft tokens, anchor tokens, and the position matrix. The length of the anchor token is set to 1. We report base and novel class accuracy and their harmonic mean (HM) averaged over 3 runs. Algorithm 1 provides AnchorOPT’s PyTorch-style pseudocode.

Algorithm 1 Pseudocode of AnchorOPT in PyTorch.

```
# anc_token: anchor token
# soft_token: soft token
# cls_token: class token
# N: length of anchor tokens
# M: length of soft tokens
# t_anc: anchor prompt
# pos_mat: position matrix
# f_t: text encoder
# f_v: image encoder

# stage 1: anchor optimization
for des_text in description_dataset:
    feat_des = f_t(des_text)
    feat_anc = f_t(t_anc)
    loss = MSE(feat_anc, feat_des)
    loss.backward()

# stage 2: adaptation
# calculate position matrix
pos_params = torch.randn(N+M, N+M)
pos_params = nn.Parameter(pos_params,
                           requires_grad=True)
pos_mat = F.gumbel_softmax(pos_params, hard=True)

for img, label in labeled_dataset:
    feat_img = f_v(img)
    t_norm = [soft_token, anc_token] * pos_mat
    feat_norm = f_t([t_norm, cls_token])

    # predictions
    l_anc = feat_img * feat_anc.t()
    l_norm = feat_img * feat_norm.t()
    l_ens = (l_anc + l_norm) / 2

    # calculate loss
    loss_kd = KL(l_norm, l_ens)
    loss_ce = CE(l_norm, label)
    loss = alpha * loss_ce + beta * loss_kd
    loss.backward()
```

S2.2.1. Category Description Generation

Inspired by previous works [31], we leverage a Large Language Model (LLM) such as GPT-3[3] to generate class-specific descriptions through the query template: “*What does a {Class} look like?*” The resulting LLM-generated descriptions are encoded into text features via a text encoder, which serves as the training objective for anchor prompts. In Table S4, we show a portion of the category descriptions generated based on the category name on the Caltech101 dataset.

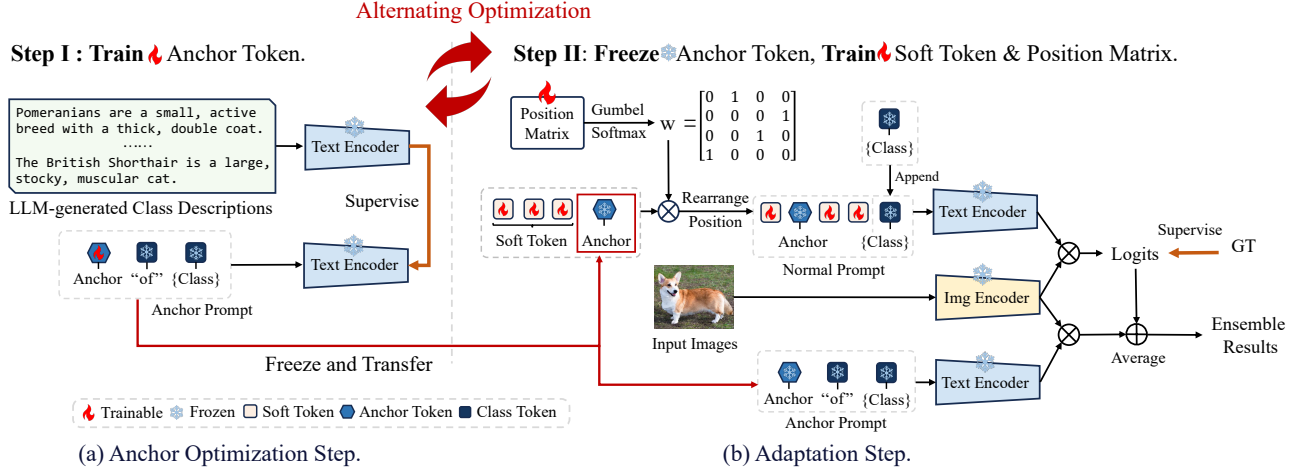


Figure S1. Illustration of one-stage training paradigm. The framework alternates between (i) optimizing anchor tokens and (ii) updating soft tokens and the position matrix while freezing anchors, iterating until convergence.

Method	Average			ImageNet			Caltech101			OxfordPets		
	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM
CoOp (IJCV 22)	82.69	63.22	71.66	76.47	67.88	71.92	98.00	89.81	93.73	93.67	95.29	94.47
ATPrompt (ICCV 25)	82.68	68.04	74.65	76.27	70.60	73.33	97.95	93.63	95.74	94.77	96.59	95.67
AnchorOPT-One Stage	80.36	75.89	78.06	75.37	71.41	73.34	97.67	95.38	96.51	94.72	98.04	96.35
AnchorOPT-Anchor Prompts	71.69	76.93	73.91	74.63	71.03	72.78	97.93	95.42	96.53	94.31	98.15	96.19
AnchorOPT-Normal Prompts	81.24	72.46	76.60	76.30	69.51	72.75	98.26	95.12	96.66	95.32	97.02	96.16
AnchorPOT-Ensemble	79.72	76.27	77.96	76.36	71.56	73.88	98.28	95.34	96.79	95.66	98.04	96.84
AnchorPOT-Final	81.24	76.27	78.68	76.30	71.56	73.85	98.26	95.34	96.78	95.32	98.04	96.66

Method	StanfordCars			Flowers102			Food101			FGVCAircraft		
	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM
CoOp (IJCV 22)	78.12	60.40	68.13	97.60	59.67	74.06	88.33	82.26	85.19	40.44	22.30	28.75
ATPrompt (ICCV 25)	77.43	66.55	71.58	97.44	67.52	79.77	88.74	87.44	88.09	40.38	27.22	32.52
AnchorOPT-One Stage	75.43	74.15	74.78	96.33	75.55	84.68	90.12	91.93	91.02	35.95	35.15	35.55
AnchorOPT-Anchor Prompts	64.57	75.27	69.50	73.41	78.86	76.04	90.04	91.62	90.82	28.87	36.65	32.30
AnchorOPT-Normal Prompts	76.73	69.68	73.04	97.22	68.13	80.12	90.40	91.23	90.81	38.48	29.42	33.35
AnchorOPT-Ensemble	74.80	74.43	74.61	92.78	75.41	84.94	90.64	91.95	91.29	36.98	35.47	36.21
AnchorOPT-Final	76.73	74.43	75.67	97.22	75.41	84.94	90.40	91.95	91.17	38.48	35.47	36.91

Method	SUN397			DTD			EuroSAT			UCF101		
	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM	Base	Novel	HM
CoOp (IJCV 22)	80.60	65.89	72.51	79.44	41.18	54.24	92.19	54.74	68.69	84.69	56.05	67.46
ATPrompt (ICCV 25)	80.84	68.64	74.24	80.83	45.49	58.22	90.34	59.79	71.96	84.49	64.96	73.45
AnchorOPT-One Stage	80.62	78.19	79.39	76.28	59.62	66.93	79.35	77.18	78.25	82.13	78.24	80.14
AnchorOPT-Anchor Prompts	74.36	78.37	76.31	55.78	62.56	58.98	57.33	80.20	66.86	75.33	78.15	76.71
AnchorOPT-Normal Prompts	79.67	75.91	77.74	74.04	57.55	64.76	83.75	66.90	74.38	83.45	76.56	79.86
AnchorOPT-Ensemble	79.42	78.73	79.07	72.11	61.72	66.51	77.34	77.14	77.24	82.50	79.18	80.81
AnchorOPT-Final	79.67	78.73	79.20	74.04	61.72	67.32	83.75	77.14	80.31	83.45	79.18	81.26

Table S1. Accuracy of each prompt in AnchorOPT.

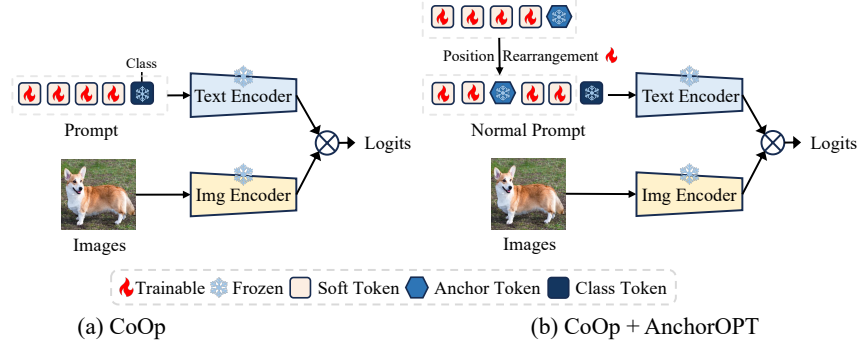


Figure S2. Architecture comparison between CoOp and CoOp+AnchorOPT. In CoOp+AnchorOPT, anchor terms are first pre-trained, then embedded into normal prompts, and multiplied with the position matrix. The final result is used as the input to the encoder.

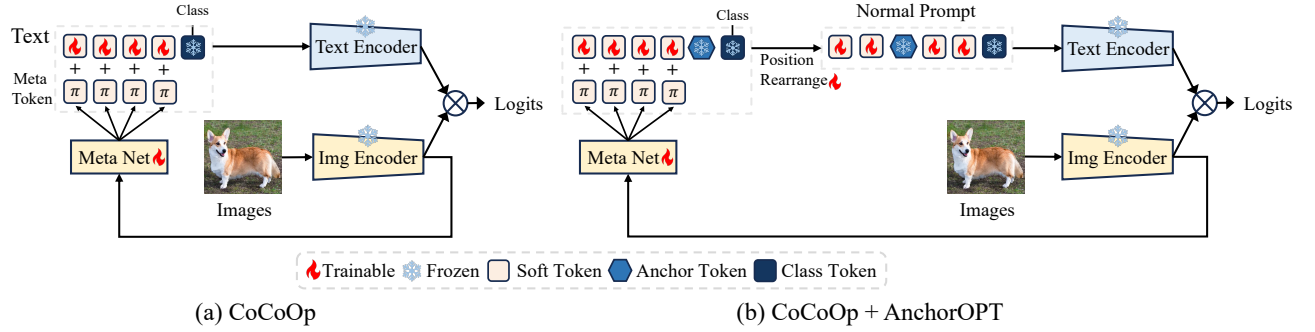


Figure S3. Architecture comparison between CoCoOp and CoCoOp+AnchorOPT. In CoCoOp+AnchorOPT, soft tokens are first augmented by the offset generated by the meta-network and subsequently multiplied by the position matrix.

S2.2.2. CoOp+AnchorOPT

A batch size of 8 and an initial learning rate of 0.0005 are employed. Models train for 100 epochs on Caltech-101, Oxford Pets, Stanford Cars, Flowers-102, FGVC Aircraft, DTD, EuroSAT, and UCF-101; 20 epochs on ImageNet, Food-101, and SUN-397. Soft token lengths are set to 4 for Caltech-101, Oxford Pets, Flowers-102, Food-101, FGVC Aircraft, DTD, and EuroSAT; 6 for ImageNet and UCF-101; and 8 for SUN-397. Anchor token length remains fixed at 1 across all datasets. Architectural integration of AnchorOPT into CoOp is illustrated in Fig. S2.

S2.2.3. CoCoOp+AnchorOPT

Consistent with the baseline, a batch size of 1 and an initial learning rate of 0.002 are used. Training spans 20 epochs for Caltech-101, Oxford Pets, Stanford Cars, Flowers-102, Food-101, FGVC Aircraft, DTD, EuroSAT, and UCF-101; 10 epochs for SUN-397 and ImageNet. Soft token lengths are set to 4 for all datasets except SUN-397 (8). Anchor token length remains uniformly 1. Integration details appear in Fig. S3. Soft tokens are processed by meta-network-generated offsets, then multiplied by the position matrix to form text encoder inputs.

S2.2.4. MaPLE+AnchorOPT

With a batch size of 4 and an initial learning rate of 0.0035, soft token lengths are set to 4 for Stanford Cars, Oxford Pets, FGVC Aircraft, SUN-397, DTD, and UCF-101; and 8 for Caltech-101, Oxford Pets, Food-101, and EuroSAT. Anchor token length remains fixed at 1. Integration is shown in Fig. S4. To preserve token diversity amid position-matrix operations (which involve selective discarding and repetition), initial text soft tokens are mapped to visual tokens before position-matrix multiplication. The resulting tokens serve as text encoder inputs, with deeper-layer soft tokens undergoing position-assigned discarding and reintroduction. The detailed comparison between the shallow and deep versions is presented in Fig. S6.

S2.2.5. DePT+AnchorOPT

Using a batch size of 4 and an initial learning rate of 0.0035, soft token lengths are set to 4 for Caltech-101, Food-101, FGVC Aircraft, and ImageNet; and 8 for Oxford Pets, Stanford Cars, Flowers-102, SUN-397, DTD, EuroSAT, and UCF-101. Anchor token length remains consistently 1. Integration details are provided in Fig. S5. Final novel-class predictions are derived by ensembling anchor prompt out-

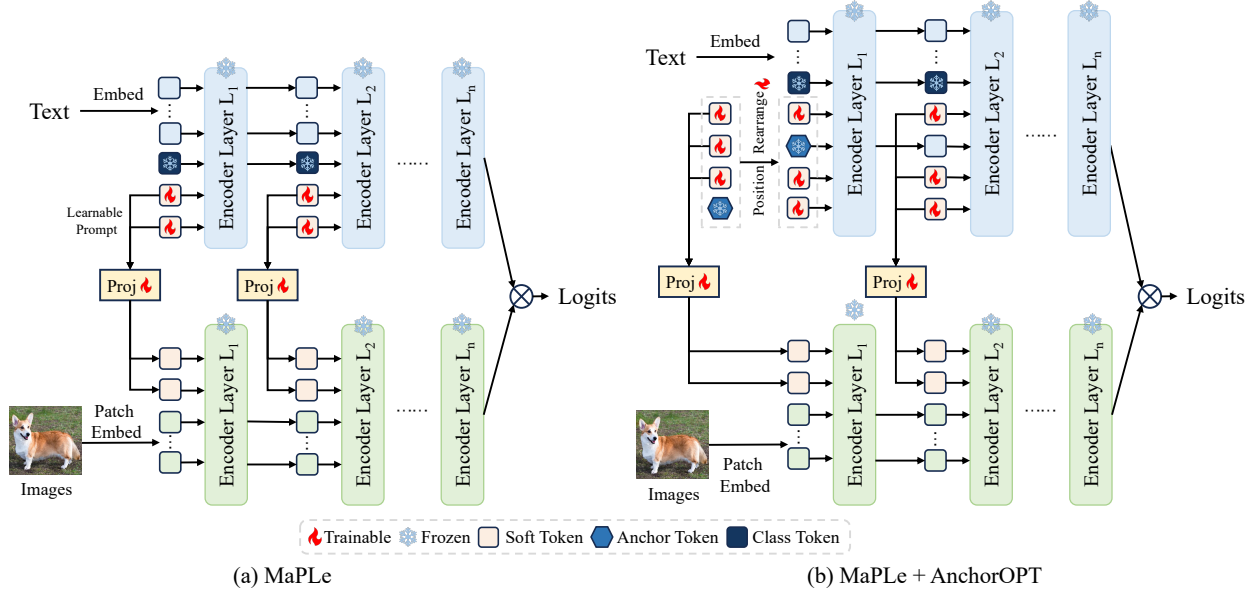


Figure S4. Architecture comparison between MaPLe and MaPLe+AnchorOPT. In MaPLe+AnchorOPT, the initial soft textual tokens are first mapped to visual tokens, which are subsequently multiplied by the position matrix. During forward propagation, the representations derived from the anchor tokens are preserved to provide semantic guidance.

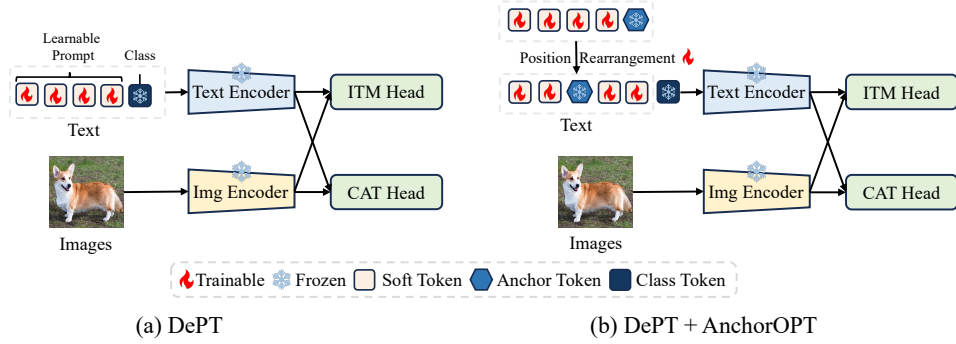


Figure S5. Architecture comparison between DePT and DePT+AnchorOPT. DePT+AnchorOPT uses the same operations as CoOp+AnchorOPT to construct the input prompt portion.

puts with results from the image-text matching (ITM) head.

S3. Experimental Results

S3.1. Base-to-Novel Experiments

Accuracy of Each Prompt. Table S1 reports the accuracy of the selected anchor prompt, normal prompt, ensemble prediction, and the final results. In this work, we integrate anchor and normal prompt predictions via equal-weighted averaging $q_{ens} = (q_{norm} + q_{anc})/2$. Equal weighting is adopted because it eliminates hyperparameter tuning while treating both prediction sources as equally reliable—a critical simplification for real-world deployment where computational efficiency and robustness are prioritized.

S3.2. Ablation Study

Proposition “None”. Table S2 presents results for anchor prompts omitting prepositions. This configuration avoids performance degradation from irrelevant tokens (e.g., “sun” or “sea”), maintaining strong generalization—though marginally inferior to preposition-augmented prompts.

Gumbel Softmax Temperature. While we fix the Gumbel Softmax temperature at 1 during primary experiments, Table S3 demonstrates that values spanning 0.1–4 yield comparable performance. This indicates model robustness to temperature variations in the Gumbel Softmax function.

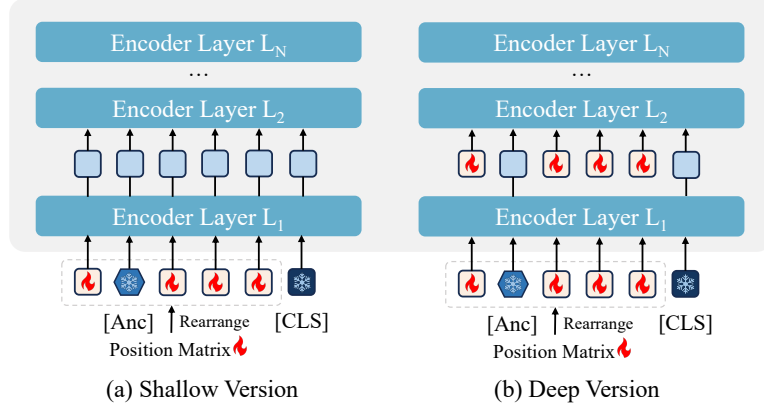


Figure S6. Architecture comparison between shallow version and deep version. In the deep version, the representations derived from the anchor tokens are preserved to provide semantic guidance.

Preposition Type	Value	Base	Novel	HM
None	None	75.87	71.43	73.58
Similar	“with”	75.96	71.49	73.66
	“at”	76.07	71.54	73.74
Irrelevant	“sun”	75.96	71.05	73.42
	“sea”	75.69	70.89	73.21
Ours	“of”	76.30	71.56	73.85

Table S2. Comparison of different preposition words on ImageNet. The absence of prepositions affects the model’s ability to generalize to base classes.

Temp	Base	Novel	HM
0.1	76.08	71.34	73.63
0.5	76.10	71.39	73.67
1	76.30	71.56	73.85
2	75.94	71.48	73.64
4	75.94	71.58	73.70

Table S3. Comparison of different temperature values of Gumbel Softmax on ImageNet. The varying temperature values have little impact on the final model performance, and the fluctuations are all within the normal range.

S4. Discussion

Visualization of Learned Anchor Tokens. To visualize learned anchor embeddings, we project embeddings into the token vocabulary space via inverse embedding. However, the resulting sequences consist of incoherent character strings that form neither valid words nor semantically meaningful content. This limitation stems from the fundamental incompatibility between the continuous latent space in which anchor tokens are optimized and the discrete tokenization process of CLIP, preventing meaningful mapping

from continuous embeddings to human-interpretable text. **Exclusion of Class Tokens During Position Rearrangement.** The class token contains Class tokens that encode task-critical semantic information essential for representation fidelity. Integrating them into position rearrangement introduces two significant challenges: (1) displacement or omission of class tokens compromises prompt efficacy and disrupts semantic representation learning; (2) increased positional flexibility elevates optimization complexity for soft prompts and anchors, degrading model performance.

S5. Future Work

Current prompt learning frameworks remain predominantly evaluated on CLIP, with limited adaptation to alternative vision-language models (VLMs) such as SigLIP and EVA-CLIP. We will address this gap by developing a cross-model benchmark to systematically evaluate and enhance prompt learning compatibility across diverse VLM architectures.

Class	Description
gerenuk	A gerenuk is a type of antelope that is native to Africa and Asia. A gerenuk is a lanky, long-necked gazelle with narrow, somewhat pointed horns that curve downward. A gerenuk is a long-necked antelope with thin legs and large ears. A gerenuk is a tall, slender antelope with long, thin legs. A gerenuk is a type of antelope with a long neck and limbs.
background	A background looks like an image or color behind the main focus of a piece.”, A background looks like the area behind the main subject of an image. A background usually has a certain color or pattern. A background is an image or color that appears behind the main content on a screen. A background generally refers to the parts of an image that are farthest from the viewer.
hawksbill	A hawksbill sea turtle has a narrow head with a hawk-like beak. A hawksbill sea turtle has a hawk-like beak, which is where it gets its name. A hawksbill looks like a small, thin turtle with a long neck and a beak-like mouth. A hawksbill looks like a small turtle with a long, pointed beak. A hawksbill sea turtle is a small to medium-sized turtle that can grow up to about 3 feet long.
headphone	A headphone typically consists of two small speakers that are attached to a headband. A headphone is a small speaker that you can wear on your head. A headphone is a small pair of speakers that fit over a person’s ears. A headphone typically consists of a pair of small speakers that are placed over the ears. Most headphones are designed to go over your ears.
ant	A ant is small, black, and has six legs. The ant is a small, black creature with six legs. A ant is a small, black insect with six legs. A typical ant is about 2. A ant typically has a dark brown or black body with a narrow waist.
butterfly	Most butterflies have brightly colored wings with intricate patterns. A butterfly typically has two large wings that are decorated with brightly colored scales. Most butterflies have brightly colored wings, with patterns made up of tiny scales. A butterfly is a flying insect. A butterfly has brightly colored wings and a long thin body.
lamp	A lamp is a household appliance that is used to illuminate a room. When most people think of a lamp, they think of a light bulb on a stand with a shade. A lamp is a household appliance that consists of a light bulb inside a housing, which is usually made of glass. A lamp looks like a small lightbulb on a metal stand. A lamp is a device that contains an electric light bulb and is used to produce light.
strawberry	A strawberry is a fruit that is generally red, although some varieties are yellow, white, or light green. A strawberry is a small, red, spherical fruit with a seed-studded surface. A strawberry is a small red fruit with a seed-covered surface. A strawberry is a small, red fruit that is covered in tiny seeds. A strawberry typically has a bright red exterior with small seeds on the surface.

Table S4. A partial display of the descriptions generated from the category names on the Caltech101 dataset.